OW-AI Platform - Feature Inventory

October 12, 2025

OW-AI Platform - Complete Feature Inventory

Executive Summary

Total Files Analyzed: 4,470+ Python files **Core Route Modules**: 34 route files **Service Modules**: 17 service modules **Data Models**: 4 model files **Discovery Status**: Phase 1 - Comprehensive mapping complete

---

🏗️ Core Application Architecture

Main Application (`main.py`)

- **Enterprise Configuration System** with graceful fallbacks
- **JWT Authentication Manager** with token validation

- **RBAC Manager** with role-based permissions
- **Health Monitoring System** with enterprise features
- **CORS Security Configuration**
- **Middleware Stack** for enterprise security
- **Route Registration System** (14+ router modules)

🔐 Authentication & Security Features

Core Authentication (`routes/auth.py` , `routes/auth_routes.py`)

- User login/logout with JWT tokens
- Password hashing (bcrypt)
- Token refresh mechanisms
- Session management

Single Sign-On (`routes/sso_routes.py`)

- SAML integration
- OIDC providers
- Enterprise SSO workflows

Enterprise User Management (`routes/enterprise_user_management_routes.py`)

- User provisioning/deprovisioning
- Role assignment

- Bulk user operations
- User lifecycle management

🛡️ Authorization & Governance Features

Authorization Center (`routes/authorization_routes.py`)

- **Multi-level authorization workflows** (1-5 approval levels)
- **Real-time policy evaluation** (sub-200ms target)
- **Risk scoring system** (0-100 scale)
- **Approval workflow management**
- **Emergency authorization procedures**
- **Audit trail generation**

MCP Governance (`routes/mcp_governance_routes.py` , `routes/unified_governance_routes.py`)

- **Model-Code-Prompt governance framework**
- **Unified governance policies**
- **MCP enterprise secure features**
- **Governance workflow integration**

Policy Management (`services/enterprise_policy_templates.py`)

- **Cedar Policy Engine integration**
- **Enterprise policy templates**

- **Policy enforcement mechanisms**
- **Cedar enforcement service**

🚨 Alert & Monitoring Features

Smart Alerts System (`routes/smart_alerts.py`)

- **Real-time alert processing**
- **Alert correlation engine**
- **Alert escalation workflows**
- **WebSocket-based real-time updates**
- **Alert monitoring service**

Alert Management (`routes/alert_routes.py` , `routes/alert_summary.py`)

- Alert creation/acknowledgment
- Alert escalation procedures
- Alert summary dashboards
- Alert lifecycle management

SIEM Integration (`routes/siem_integration.py` , `routes/siem_simple.py`)

- Security Information Event Management
- Log aggregation
- Threat detection

- Incident response workflows

🤖 AI & Rule Engine Features

Smart Rules Engine (`routes/smart_rules_routes.py`)

- **Natural language rule creation**
- **AI-powered rule generation**
- **A/B testing framework**
- **Rule performance analytics**
- **Dynamic rule evaluation**

Agent Management (`routes/agent_routes.py`)

- Agent registration
- Agent action tracking
- Agent performance monitoring
- Agent workflow integration

📊 Analytics & Reporting Features

Analytics Dashboard (`routes/analytics_routes.py`)

- Performance metrics

- Usage analytics
- Security analytics
- Business intelligence dashboards

Audit & Compliance (`routes/audit_routes.py`)

- **Immutable audit trails** (`services/immutable_audit_service.py`)
- Compliance reporting
- Regulatory framework mapping
- Audit log management

🔧 Administrative Features

Admin Console (`routes/admin_routes.py`)

- System configuration
- User management
- System monitoring
- Maintenance operations

Support System (`routes/support_routes.py`)

- Ticket management
- User support workflows
- System diagnostics
- Help desk integration

🔐 Enterprise Security Features

Secrets Management (`routes/enterprise_secrets_routes.py`)

- Secure secret storage
- Secret rotation
- Access control for secrets
- Encryption key management

Data Rights Management (`routes/data_rights_routes.py`)

- **GDPR compliance features**
- Data subject rights
- Data portability
- Right to be forgotten

🤝 Integration & Orchestration

Automation Orchestration (`routes/automation_orchestration_routes.py`)

- Workflow automation
- Process orchestration
- Integration management

- Event-driven automation

Security Bridge (`services/security_bridge_service.py`)

- External system integration
- Security protocol bridging
- Cross-platform communication

📈 Compliance & Risk Management

NIST Framework Integration (`services/nist_mapper.py`)

- NIST control mapping
- Compliance assessment
- Security control implementation

MITRE ATT&CK Integration (`services/mitre_mapper.py`)

- Threat technique mapping
- Attack pattern analysis
- Defense strategy alignment

CVSS Calculator (`services/cvss_calculator.py` , `services/cvss_auto_mapper.py`)

- Vulnerability scoring
- Risk assessment automation

- Security rating system

💾 Data Management

Core Models (`models.py`)

- User management
- Alert system
- Agent actions
- Audit trails
- Authorization workflows

Audit Models (`models_audit.py`)

- Audit trail structures
- Compliance data models
- Regulatory reporting models

Data Rights Models (`models_data_rights.py`)

- GDPR data structures
- Privacy management models
- Data subject tracking

MCP Governance Models (`models_mcp_governance.py`)

- Governance policy models

- MCP workflow structures
- Enterprise governance data

🔄 Workflow & Process Management

Workflow Services

- **Workflow Bridge** (`services/workflow_bridge.py`)
- **Approver Selector** (`services/approver_selector.py`)
- **Workflow Approver Service** (`services/workflow_approver_service.py`)
- **SLA Monitor** (`services/sla_monitor.py`)

Condition Engine (`services/condition_engine.py`)

- Business rule evaluation
- Conditional logic processing
- Dynamic decision making

Action Taxonomy (`services/action_taxonomy.py`)

- Action classification
- Taxonomy management
- Action standardization

🏥 System Health & Monitoring

Health System (`health.py` - Enterprise Module)

- System health monitoring
- Performance metrics
- Availability tracking
- Enterprise health dashboards

Logging System (`routes/log_routes.py`)

- Centralized logging
- Log analysis
- Log retention policies
- Log correlation

📋 Feature Categorization Summary

✅ Core Features (Must Test)

1. **Authentication System** (Login, JWT, Password Management)
2. **Authorization Center** (Multi-level approvals, Risk scoring)
3. **Smart Rules Engine** (AI generation, A/B testing)
4. **Alert Management** (Real-time processing, Escalation)
5. **Policy Management** (Cedar engine, Enterprise templates)

🔧 Enterprise Features (Must Test)

1. **SSO Integration** (SAML, OIDC)
2. **Governance Framework** (MCP, Unified policies)
3. **Compliance Systems** (NIST, MITRE, CVSS)
4. **Audit Trails** (Immutable logging)
5. **Data Rights Management** (GDPR compliance)

📊 Analytics Features (Must Test)

1. **Performance Analytics**
2. **Security Analytics**
3. **Business Intelligence**
4. **Compliance Reporting**

🤝 Integration Features (Must Test)

1. **SIEM Integration**
2. **Automation Orchestration**
3. **External API Bridges**
4. **Workflow Integration**

🎯 Phase 2 Testing Strategy

High Priority Testing (Critical Business Features)

1. **End-to-end authentication flows**

2. **Authorization workflows (1-5 levels)**
3. **Alert processing and escalation**
4. **Rule creation and execution**
5. **Policy evaluation performance**

Medium Priority Testing (Enterprise Features)

1. **SSO integration workflows**
2. **Governance policy enforcement**
3. **Compliance framework mapping**
4. **Audit trail integrity**
5. **Data rights management**

Low Priority Testing (Support Features)

1. **Analytics dashboard functionality**
2. **Admin console operations**
3. **Support system workflows**
4. **Integration bridges**

Next Phase: Begin comprehensive feature testing with evidence collection and database verification.

Estimated Testing Time: 8-12 hours for complete end-to-end validation **Priority**: Focus on business-critical authorization and alert features first